

BENABOU Hafsa COMPAORE Moustapha DIALLO Mamadou Tahirou

Institut National des Postes et Télécommunications
Filière Sciences de Données – 2ème année Ingénieur

Juin 2026



Plan de la présentation

- ➊ Contexte et objectif
- ➋ Données utilisées
- ➌ Architecture générale
- ➍ Classification d'intentions
- ➎ Mapping MCP et agent CRM
- ➏ API, interface et déploiement
- ➐ Évaluation
- ➑ Bilan et perspectives

Question de départ

Comment automatiser une partie du support client tout en gardant une réponse structurée, traçable et exploitable par un système CRM ?

Besoins côté client

- ▶ annuler ou suivre une commande ;
- ▶ demander une facture ou un remboursement ;
- ▶ modifier une adresse ;
- ▶ contacter un agent humain.

Besoins côté système

- ▶ comprendre l'intention ;
- ▶ appeler le bon outil métier ;
- ▶ produire une réponse claire ;
- ▶ fonctionner sur une machine CPU.

Construire un agent CRM modulaire capable de traiter une demande client de bout en bout.

Pipeline cible

Message utilisateur → classification de l'intention → sélection du tool MCP → exécution CRM → réponse finale

- ▶ Une solution reproductible : scripts, API, interface et documentation.
- ▶ Une solution pragmatique : modèle léger TF-IDF + LogisticRegression sur CPU.
- ▶ Une solution extensible : variante Transformer et reformulation optionnelle avec Ollama.

Choix du dataset

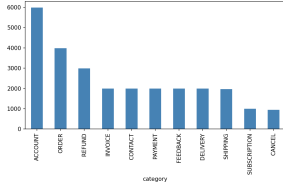
Dataset	Utilité	Décision
Telco Customer Churn	Prédiction du churn client	Intéressant pour un futur tool de scoring, mais pas centré sur les messages.
92k Call Center Scripts	Corpus conversationnel riche	Utile pour RAG ou mémoire, mais moins direct pour la classification d'intentions.
Bitext Customer Support Chat-bot	26 900 paires instruction/réponse, 27 intentions	Retenu comme dataset principal du projet.

Pourquoi Bitext ?

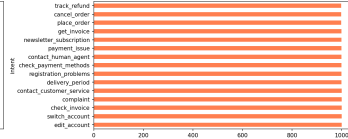
Il correspond directement au besoin : entraîner un classificateur d'intentions et router chaque intention vers une action CRM.

Analyse exploratoire

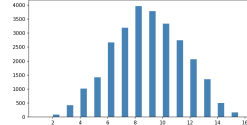
Distribution des categories



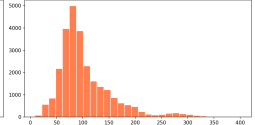
Top 15 intents



Distribution longueur des instructions



Distribution longueur des reponses



- ▶ analyse des intentions et catégories ;
- ▶ nettoyage des templates ;
- ▶ splits train / validation / test ;
- ▶ génération des fichiers CSV et JSONL.

Composants

- ▶ data/ : dataset et artefacts ;
- ▶ src/ : logique métier ;
- ▶ scripts/ : préparation, entraînement, évaluation ;
- ▶ api.py : service FastAPI ;
- ▶ streamlit_app.py : interface de démonstration.

Flux

- 1 Préparer le dataset Bitext.
- 2 Entraîner le classificateur.
- 3 Mapper l'intention vers un tool MCP.
- 4 Orchestrer la réponse via l'agent.
- 5 Exposer le tout via API et Streamlit.

Modèle principal

- ▶ TF-IDF avec unigrammes et bigrammes ;
- ▶ maximum de 10 000 features ;
- ▶ LogisticRegression ;
- ▶ sauvegarde dans `data/intent_classifier.pkl`.

Pourquoi ce choix ?

- ▶ rapide à entraîner ;
- ▶ robuste comme baseline ;
- ▶ adapté aux contraintes CPU.

Variante optionnelle

- ▶ mini-BERT :
google/bert_uncased_L-2_H-128_A-2 ;
- ▶ DistilBERT possible ;
- ▶ plus coûteux que TF-IDF sur CPU.

Mapping MCP : de l'intention vers l'action

Intention	Tool MCP appelé	Paramètres
cancel_order	cancel_order	order_id
track_order	get_order_status	order_id
change_shipping_address	update_shipping_address	order_id, new_address
get_refund	process_refund	order_id, reason
contact_human_agent	escalate_to_human	reason

Idée clé

Le modèle ne décide pas directement d'une réponse finale : il choisit une intention, puis l'agent appelle un tool métier explicite.

- 1 Recevoir le message utilisateur.
- 2 Prédire l'intention et la confiance.
- 3 Appliquer un fallback si la confiance est faible.
- 4 Extraire les paramètres simples.
- 5 Exécuter le tool MCP.
- 6 Générer ou retourner la réponse.

Exemple

I need to cancel my order #12345

Intent : cancel_order

Tool : cancel_order(order_id)

Réponse : confirmation structurée ou reformulation Ollama.

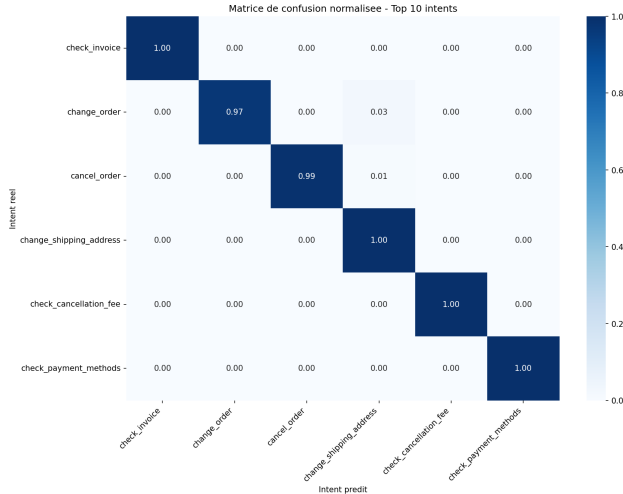
API FastAPI

- ▶ GET /health
- ▶ POST /chat
- ▶ DELETE /session/{session_id}
- ▶ GET /intents

Les sessions sont gardées en mémoire pour conserver un historique conversationnel.

Interface Streamlit

- ▶ test d'un message client ;
- ▶ affichage de l'intention et de la confiance ;
- ▶ visualisation du tool MCP appelé ;
- ▶ dashboard de session ;
- ▶ export de l'historique.



Mesures prévues

- ▶ accuracy globale ;
- ▶ confiance moyenne ;
- ▶ ratio de prédictions avec confiance > 80% ;
- ▶ rapport de classification ;
- ▶ matrice de confusion.

Les tests fonctionnels couvrent le classificateur, le mapping MCP, l'API, Streamlit et les cas limites.

Scénario court à présenter

- ❶ Lancer l'API FastAPI.
- ❷ Ouvrir l'interface Streamlit.
- ❸ Envoyer : I want to cancel my order number 12345.
- ❹ Montrer l'intention `cancel_order`, le score de confiance et le tool appelé.
- ❺ Tester un message ambigu pour montrer l'escalade vers un humain.

Commande API :

```
uvicorn api:app -reload -port 8000
```

Commande interface :

```
streamlit run streamlit_app.py
```

Réalisé

- ▶ projet Python modulaire ;
- ▶ workflow dataset reproductible ;
- ▶ classificateur TF-IDF entraînable ;
- ▶ mapping MCP complet ;
- ▶ agent CRM fonctionnel ;
- ▶ API FastAPI et interface Streamlit ;
- ▶ mode Ollama et mode sans Ollama.

Limites

- ▶ tools CRM simulés ;
- ▶ extraction de paramètres simple ;
- ▶ sessions en mémoire ;
- ▶ intentions multiples non gérées ;
- ▶ Transformer encore optionnel.

- ▶ Connecter les tools MCP à un vrai backend CRM.
- ▶ Ajouter une persistance Redis ou SQLite pour les sessions.
- ▶ Améliorer l'extraction d'entités : numéro de commande, adresse, email, raison.
- ▶ Gérer les demandes contenant plusieurs intentions.
- ▶ Intégrer un tool `predict_churn(customer_id)` avec Telco Churn.
- ▶ Comparer TF-IDF, Transformer et embeddings sur les mêmes métriques.

Conclusion

Le projet démontre une solution agentique CRM locale, modulaire et extensible, adaptée aux contraintes matérielles du mini-projet.

Merci pour votre attention

Questions ?

